

How to fix Git error: object file is empty?

Asked 8 years, 3 months ago Active 1 month ago Viewed 194k times

When I try to commit changes, I get this error:

478

```
error: object file .git/objects/31/65329bb680e30595f242b7c4d8406ca63eeab0 is empty
fatal: loose object 3165329bb680e30595f242b7c4d8406ca63eeab0 (stored in
.git/objects/31/65329bb680e30595f242b7c4d8406ca63eeab0) is corrupt
```



235

I tried `git fsck` I've got:



```
error: object file .git/objects/03/dfd60a4809a3ba7023cbf098eb322d08630b71 is empty
fatal: loose object 03dfd60a4809a3ba7023cbf098eb322d08630b71 (stored in
.git/objects/03/dfd60a4809a3ba7023cbf098eb322d08630b71) is corrupt
```

Any idea how to solve this error ?

git

edited Sep 17 at 19:49



BuZZ-dEE

3,285 ● 6 ● 44 ● 66

asked Jul 29 '12 at 2:38



simo

19.6k ● 30 ● 101 ● 177

▲ Did you forcibly kill a `git add` operation? Is your hard disk full? — [cdhowie](#) Jul 29 '12 at 2:46



▲ No, my hard disk is not full, I don't remember that I forcibly killed a git add operation, what if I did ? how can I solve this ? — [simo](#) Jul 29 '12 at 3:23



▲ no, the error is still there ... — [simo](#) Jul 30 '12 at 2:17



2 ▲ If this repository exists on a remote repository, you could try copying that file from there to your local one if exists on your remote repository. — [Attila Szeremi](#) Aug 10 '12 at 8:59



2 ▲ I got this error when my permissions in the `.git` directory got screwed up somehow and I didn't have read access. So it can happen in cases where the files aren't empty but they just can't be written to. Fixing permissions and running `git fsck` took care of it. — [Jake Anderson](#) Jan 17 '15 at 6:03



24 Answers

Active

Oldest

Votes



I had a similar problem. My laptop ran out of battery during a git operation. Boo.

944

I didn't have any backups. (N.B. Ubuntu One is not a backup solution for git; it will helpfully overwrite your sane repository with your corrupted one.)



To the OP: if this was a bad way to fix it, please leave a comment. It did, however, work for me.

By using our site, you acknowledge that you have read and understand our [Cookie Policy](#), [Privacy Policy](#), and our [Terms of Service](#).



+100
Step 1: Make a backup of .git (in fact I do this in between every step that changes something, but with a new copy-to name, e.g. .git-old-1, .git-old-2, etc.):

```
cp -a .git .git-old
```

Step 2: Run `git fsck --full`

```
nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ git fsck --full
error: object file .git/objects/8b/61d0135d3195966b443f6c73fb68466264c68e is empty
fatal: loose object 8b61d0135d3195966b443f6c73fb68466264c68e (stored in
.git/objects/8b/61d0135d3195966b443f6c73fb68466264c68e) is corrupt
```

Step 3: Remove the empty file. I figured what the heck; its blank anyway.

```
nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ rm
.git/objects/8b/61d0135d3195966b443f6c73fb68466264c68e
rm: remove write-protected regular empty file `.git/objects/8b/61d0135d3195966b443f6c73fb68466264c68e'? y
```

Step 3: Run `git fsck` again. Continue deleting the empty files. You can also `cd` into the `.git` directory and run `find . -type f -empty -delete -print` to remove all empty files. Eventually git started telling me it was actually doing something with the object directories:

```
nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ git fsck --full
Checking object directories: 100% (256/256), done.
error: object file .git/objects/e0/cbccee33aea970f4887194047141f79a363636 is empty
fatal: loose object e0cbccee33aea970f4887194047141f79a363636 (stored in
.git/objects/e0/cbccee33aea970f4887194047141f79a363636) is corrupt
```

Step 4: After deleting all of the empty files, I eventually came to `git fsck` actually running:

```
nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ git fsck --full
Checking object directories: 100% (256/256), done.
error: HEAD: invalid sha1 pointer af9fc0c5939eee40f6be2ed66381d74ec2be895f
error: refs/heads/master does not point to a valid object!
error: refs/heads/master.u1conflict does not point to a valid object!
error: 0e31469d372551bb2f51a186fa32795e39f94d5c: invalid sha1 pointer in cache-tree
dangling blob 03511c9868b5dbac4ef1343956776ac508c7c2a2
missing blob 8b61d0135d3195966b443f6c73fb68466264c68e
missing blob e89896b1282fbae6cf046bf21b62dd275aaa32f4
dangling blob dd09f7f1f033632b7ef90876d6802f5b5fed79a
missing blob caab8e3d18f2b8c8947f79af7885cdeeeae192fd
missing blob e4cf65ddf80338d50ecd4abcf1caf1de3127c229
```

Step 5: Try `git reflog`. Fail because my HEAD is broken.

```
nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ git reflog
fatal: bad object HEAD
```

Step 6: Google. Find [this](#). Manually get the last two lines of the reflog:

```
nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ tail -n 2 .git/logs/refs/heads/master
f2d4c4868ec7719317a8fce9dc18c4f2e00ede04 9f0ahf890b113a287e10d56b66dhah66adc1662d Nathan.VanHoudnos
```



Step 7: Note that from Step 6 we learned that the HEAD is currently pointing to the very last commit. So let's try to just look at the parent commit:

```
nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ git show 9f0abf890b113a287e10d56b66dbab66adc1662d
commit 9f0abf890b113a287e10d56b66dbab66adc1662d
Author: Nathan VanHoudnos <nathanvan@XXXXXX>
Date: Mon Sep 10 15:56:17 2012 -0400

    up to p. 24, including correcting spelling of my name

diff --git a/tex/MCMC-in-IRT.tex b/tex/MCMC-in-IRT.tex
index 86e67a1..b860686 100644
--- a/tex/MCMC-in-IRT.tex
+++ b/tex/MCMC-in-IRT.tex
```

It worked!

Step 8: So now we need to point HEAD to 9f0abf890b113a287e10d56b66dbab66adc1662d.

```
nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ git update-ref HEAD
9f0abf890b113a287e10d56b66dbab66adc1662d
```

Which didn't complain.

Step 9: See what fsck says:

```
nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ git fsck --full
Checking object directories: 100% (256/256), done.
error: refs/heads/master.ui\conflict does not point to a valid object!
error: 0e31469d372551bb2f51a186fa32795e39f94d5c: invalid sha1 pointer in cache-tree
dangling blob 03511c9868b5dbac4ef1343956776ac508c7c2a2
missing blob 8b61d0135d3195966b443f6c73fb68466264c68e
missing blob e89896b1282fbae6cf046bf21b62dd275aaa32f4
dangling blob dd09f7f1f033632b7ef90876d6802f5b5fede79a
missing blob caab8e3d18f2b8c8947f79af7885cdeeeae192fd
missing blob e4cf65ddf80338d50ecd4abcf1caf1de3127c229
```

Step 10: The invalid sha1 pointer in cache-tree seemed like it was from a (now outdated) index file ([source](#)). So I killed it and reset the repo.

```
nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ rm .git/index
nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ git reset
Unstaged changes after reset:
M   tex/MCMC-in-IRT.tex
M   tex/recipe-example/build-example-plots.R
M   tex/recipe-example/build-failure-plots.R
```

Step 11: Looking at the fsck again...

```
nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ git fsck --full
Checking object directories: 100% (256/256), done.
error: refs/heads/master.ui\conflict does not point to a valid object!
dangling blob 03511c9868b5dbac4ef1343956776ac508c7c2a2
dangling blob dd09f7f1f033632b7ef90876d6802f5b5fede79a
```

The [dangling blobs are not errors](#). I'm not concerned with master.u1conflict, and now that it is working I don't want to touch it anymore!

Step 12: Catching up with my local edits:

```
nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ git status
# On branch master
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
#   modified:   tex/MCMC-in-IRT.tex
#   modified:   tex/recipe-example/build-example-plots.R
#   modified:   tex/recipe-example/build-failure-plots.R
#
< ... snip ... >
no changes added to commit (use "git add" and/or "git commit -a")

nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ git commit -a -m "recovering from the git fiasco"
[master 7922876] recovering from the git fiasco
 3 files changed, 12 insertions(+), 94 deletions(-)

nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ git add tex/sept2012_code/example-code-testing.R
nathanvan@nathanvan-N61Jq:~/workspace/mcmc-chapter$ git commit -a -m "adding in the example code"
[master 385c023] adding in the example code
 1 file changed, 331 insertions(+)
 create mode 100644 tex/sept2012_code/example-code-testing.R
```

So hopefully that can be of some use to people in the future. I'm glad it worked.

edited May 23 '17 at 10:31



Community ♦

1 ● 1

answered Sep 11 '12 at 13:53



Nathan VanHoudnos

10.8k ● 2 ● 20 ● 27

- 93 ▲ Excellent answer, together with all the steps and all. I suppose you saved me from googling each and every one of those! – [Zlatko](#) Oct 28 '12 at 8:58
- 28 ▲ Worked like a charm! I wish I could upvote this multiple times ;) – [MarcDefiant](#) Apr 15 '13 at 18:54
- 1 ▲ Hmm, my laptop died during a git operation (SparkleShare was trying to commit my notes as it died) and after that the repo was corrupted in this way. I followed your steps up till 6 but it seems like the last several commits were actually supposed to be part of the empty object files that I deleted? In fact the last 3 commits were basically completely broken so I guess there was nothing I could do. Luckily I don't actually need the individual commits from SparkleShare and I can just copy over the dirty file from one machine to another and merge. – [Ibrahim](#) Apr 22 '13 at 20:41
- 15 ▲ Awesome, awesome answer. Thanks especially for including your reasoning behind each step. You've saved my repo! (Well, I still have a 'bad ref for refs/heads/master' error from fsck, but that is relatively light.) – [Jon Carter](#) May 31 '13 at 21:06
- 3 ▲ Thanks, I learned a lot about some of the git functionality whilst also saving my bacon on an important commit! Coincidentally it was also because of low battery on the laptop. – [HarbyUK](#) Jul 2 '14 at 10:34

▲ The git object files have gone corrupt (as pointed out in other answers as well). This can happen during machine crashes, etc.





I had the same thing. After reading the other top answers here I found the quickest way to fix the broken git repository with the following commands (execute in the git working directory that contains the `.git` folder):

(Be sure to back up your git repository folder first!)

```
find .git/objects/ -type f -empty | xargs rm
git fetch -p
git fsck --full
```

This will first **remove any empty object files** that cause corruption of the repository as a whole, and then **fetch down the missing objects** (as well as latest changes) from the remote repository, and then do a full **object store check**. Which, at this point, should succeed without any errors (there may be still some warnings though!)

PS. This answer suggests you have a remote copy of your git repository somewhere (e.g. on GitHub) and the broken repository is the local repository that is tied to the remote repository which is still in tact. If that is not the case, then do not attempt to fix it the way I recommend.

answered Jun 29 '15 at 7:35



[Martin Tajur](#)

2,894 ● 1 ● 16 ● 13

▲ Thanks for this, I pretty religiously push to my remote branches and your solution worked for me. I tried @mCorr's below first but after I committed the new files the repo reverted back to being corrupted. This approach solved it – [mr_than](#) Sep 12 '15 at 9:33

▲ as most of have a remote. This solution is really clean and sufficient. – [jackOfAll](#) Jan 23 '17 at 7:15

7 ▲ Had this problem after shutting down a VM in which I was working with a git repo. This solution worked perfectly. – [Giscard Biamby](#) Mar 21 '17 at 23:59

▲ Thanks Martin, excellent and compact solution. Although I might put that PS first, with the warning in bold, to prevent naive users trying the same on a just-local setup. Like Giscard, this arises for me when shutting down a VM... will update if I find a more permanent solution than doing this each time. – [thclark](#) Apr 3 '17 at 11:10

3 ▲ A VM crush did this to my local git repo aswell, i can confirm that this solution is 100% working in that case – [Amin.T](#) Apr 18 '17 at 10:58

|

This error happens to me when I am pushing my commit and my computer hangs. This is how I've fix it.

36

Steps to fix

```
git status
```

show the empty/corrupt object file



remove it

```
git status
```

I got `fatal: bad object HEAD` message

```
rm .git/index
```

I remove the `index` for the reset

```
git reset
```

fatal: Could not parse object 'HEAD'.

```
git status  
git pull
```

just to check whats happening

```
tail -n 2 .git/logs/refs/heads/MY-CURRENT-BRANCH
```

prints the last 2 lines `tail -n 2` of log branch to show my last 2 `commit hash`

```
git update-ref HEAD 7221fa02cb627470db163826da4265609aba47b2
```

I pick the last `commit hash`

```
git status
```

shows all my file as `deleted` because i removed the `.git/index` file

```
git reset
```

continue to the reset

```
git status
```

verify my fix

Note: The steps starts when I landed on this question and used the answers as reference.

answered May 5 '15 at 2:34



marlo

3 785 3 19 28



34 I solved this removing the various empty files that git fsck was detecting, and then running a simple git pull.

I find it disappointing that now that even filesystems implement journaling and other "transactional" techniques to keep the fs sane, git can get to a corrupted state (and not be able to recover by itself) because of a power failure or space on device.



answered May 7 '13 at 14:10



Simone Gianni
10.3k ● 33 ● 45

- 3 I'm sure that the answer above is technically better, but it stopped working at step 6 and was way above my head technically. The expedient approach is git pull – [mblackwell8](#) Oct 24 '13 at 1:18
- 2 I encountered a situation where, after doing steps 1-11 of the instructions from Nathan's answer (which worked great!), I had an error that was saying refs/origin/master and refs/origin/head were not defined (or something like that). git pull fixed that. So I think both solutions work together. – [bchurchill](#) Apr 28 '14 at 6:00
- 2 I'm aware that the filesystems used are normally journaling the meta-data only. You can turn on journaling for the data as well, but I guess its not default due to the overhead(?) That's probably why the files were empty.... and filesystems usually observe transactions per file, whereas git has multiple files being modified per transaction, thus even if the fs keeps consistency per file, if git doesn't, then I guess git will result in inconsistent states... – [Heartinpiece](#) Aug 19 '15 at 1:09
- ▲ This answer worked for me, others are way to complicated. – [ldog](#) Sep 8 '18 at 6:43
- 1 Way faster solution than the accepted answer. To everyone who has scrolled this far, follow this answer if you are in a hurry. – [Sri Harsha Kappala](#) May 4 at 17:40

10 I just had the same issue : after pulling the distant repository, when I did a git status I got : "error: object file (...) is empty" "fatal: loose object (...) is corrupted"

The way I resolved this was to :



1. git stash
2. removing git file in error (not sure that is necessary)
3. git stash clear

I dont know exactly what things happened, but that instructions seemed to make everything clean.

answered Feb 13 '13 at 9:55



Nicolas
101 ● 1 ● 3

- 2 I always like the simpler answers :) For Step 2 here I used the command provided in @Nathan VanHoudnos 's answer: `cd .git/ && find . -type f -empty -delete` – [mopo922](#) Dec 9 '14 at 16:13

8 Because I have to reboot my VM regularly, so somehow this problem happens to me very often. After few times of it, I realized I cannot repeat the process described by @Nathan-Vanhoudnos every time this happens, though it always works. Then I figured out the following faster solution.





Move your entire repo to another folder.

```
mv current_repo temp_repo
```

Step 2

Clone the repo from origin again.

```
git clone source_to_current_repo.git
```

Step 3

Remove **Everything** under the new repo except the **.git** folder.

Step 4

Move **Everything** from the **temp_repo** to the new repo **except** the **.git** folder.

Step 5

Remove the **temp_repo**, and we are done.

After few times, I'm sure you can do this procedures very quickly.

answered Dec 19 '15 at 0:58



[haoqiang](#)

111 ● 1 ● 4

2 ▲ Or don't move your current repo, 1) create a new clone `git clone source_to_current_repo.git clean_repo`, 2) backup the old `.git` folder, 3) copy over the clean `.git` folder. – [moi](#) May 10 '16 at 12:53

▲ You are right. I already did it now. Will edit the answer later. – [haoqiang](#) May 11 '16 at 9:05

▲ @haoqiang: You wrote "Because I have to reboot my VM regularly, so somehow this problem happens to me very often." We experience the same. Have you made any progress on the root cause - VM settings that makes the problem happen less often? – [hansfn](#) Mar 27 at 17:05



6



1. mv your folder app to make backup, i.e. `mv app_folder app_folder_bk` (it is like a **git stash**)

2. `git clone your_repository`

3. Finally,. Open a merge tool (I use meld diff viewer linux or Winmerge Windows) and copy the changes from right(**app_folder_bk**) to left(new **app_folder**) (it is like a **git stash apply**).

That's all. Maybe it is not the best way, but I think it is so practical .

edited Feb 20 '14 at 15:35



[OlivierLi](#)

2,498 ● 19 ● 29

answered Feb 20 '14 at 15:15



[cyberfranco](#)

61 ● 1 ● 2



```
git stash
git checkout master
cd .git/ && find . -type f -empty -delete
git branch your-branch-name -D
git checkout -b your-branch-name
git stash pop
```

resolve my problem

edited Jul 13 '18 at 5:53



david

3,207 ● 9 ● 26 ● 40

answered Jul 13 '18 at 4:02



Gui-yi

41 ● 1

▲ this helped. Thank you. – [swateek](#) Dec 7 '18 at 6:15



▲ If the repo is clean, you just need to "cd .git/ && find . -type f -empty -delete && cd - && git pull", you may need to checkout some files in `git status` since some of the are empty. – [CodyChan](#) Jun 21 '19 at 6:15

▲ In my case, this error occurred because I was typing the commit message and my notebook turned off.

3

I did these steps to fix the error:

- `git checkout -b backup-branch` # Create a backup branch
- `git reset --hard HEAD~4` # Reset to the commit where everything works well. In my case, I had to back 4 commits in the head, that is until my head be at the point before I was typing the commit message.
Before doing this step, copy the hash of the commits you will reset, in my case I copied the hash of the 4 last commits
- `git cherry-pick <commit-hash>` # Cherry pick the reseted commits (in my case are 4 commits, so I did this step 4 times) from the old branch to the new branch.
- `git push origin backup-branch` # Push the new branch to be sure everything works well
- `git branch -D your-branch` # Delete the branch locally ('your-branch' is the branch with problem)
- `git push origin :your-branch` # Delete the branch from remote
- `git branch -m backup-branch your-branch` # Rename the backup branch to have the name of the branch that had the problem
- `git push origin your-branch` # Push the new branch
- `git push origin :backup-branch` # Delete the backup branch from remote

edited Jun 2 '18 at 3:30

answered Apr 27 '14 at 18:37



androidevil

8,060 ● 12 ● 37 ● 77

▲ Here is a really simple and quick way to deal with this problem **IF** you have a local repo with all the branches

By using our site, you acknowledge that you have read and understand our [Cookie Policy](#), [Privacy Policy](#), and our [Terms of Service](#).





1. Create a new empty repo on the server (or delete the old repo and create a new one in its place)
2. Change the remote URL of your local copy to point to the remote URL of the new repo.
3. Push all branches from your local repo to the new server repo.

This preserves all the commit history and branches that you had in your local repo.

If you have collaborators on the repo, then I think in many cases all your collaborators have to do is change the remote URL of their local repo as well, and optionally push any commits they have that the server doesn't have.

This solution worked for me when I ran into this same problem. I had one collaborator. After I pushed my local repo to the new remote repo, he simply changed his local repo to point to the remote repo URL and everything worked fine.

edited Oct 30 '14 at 16:13

answered Oct 30 '14 at 16:08



David French

141 ● 1 ● 6



I run into this problem a lot with virtual machines.

2

For me the following works:



```
cd /path/to/your/project  
rm -rf .git
```

If you want to save yourself some downloads - go in your file explorer and delete all files in the folder that are already committed and leave in your /vendor and /node_modules (I work with composer and npm) folders.

then just create a new repo

```
git init
```

add your remote

```
git remote add origin ssh://git@github.com/YourUsername/repoName.git
```

and fetch the branch / all of it

```
git fetch origin somebranch
```

and check it out

```
git checkout somebranch
```

then you should be at the point before the error.



Regards.

answered Feb 2 at 5:37



germanyDevelops
29 ● 2

Let's go simple.. only case you uploaded source to remote git repo

2

1. Backup your .git

2. check your git



```
git fsck --full
```

3. remove empty object file (all)

```
rm .git/objects/8b/61d0135d3195966b443f6c73fb68466264c68e
```

4. check your git again.

```
git fsck --full
```

5. pull your source from remote git

```
git pull origin master
```

edited Apr 2 '18 at 4:54



phucv
23.8k ● 11 ● 87 ● 325

answered Sep 13 '17 at 9:48



AXT.SIREN
21 ● 1

2

I am assuming you have a remote with all relevant changes already pushed to it. I did not care about local changes and simply wanted to avoid deleting and recloning a large repository. If you do have important local changes you might want to be more careful.



I had the same problem after my laptop crashed. Probably because it was a large repository I had quite a few corrupt object files, which only appeared one at a time when calling `git fsck --full`, so I wrote a small shell one-liner to automatically delete one of them:

```
$ sudo rm `git fsck --full 2>&1 | grep -oE -m 1 ".git/objects/[0-9a-f]{2}/[0-9a-f]*"`
```

- `2>&1` redirects the error message to stdout to be able to grep it
- grep options used:
 - `-o` only returns the part of a line that actually matches
 - `-E` enables advanced regexes
 - `-m 1` make sure only the first match is returned

• `-oE` matches any of the characters between 0 and 9 and a and f if two of them occur



- `[0-9a-f]*` matches any number of the characters between 0 and 9 and a and f occurring together

It still only deletes one file at a time, so you might want to call it in a loop like:

```
$ while true; do sudo rm `git fsck --full 2>&1 | grep -oE -m 1 ".git/objects/[0-9a-f]{2}/[0-9a-f]*"`; done
```

The problem with this is, that it does not output anything useful anymore so you do not know when it is finished (it should just not do anything useful after some time)

To "fix" this I then just added a call of `git fsck --full` after each round like so: `$ while true; do sudo rm `git fsck --full 2>&1 | grep -oE -m 1 ".git/objects/[0-9a-f]{2}/[0-9a-f]*"`; git fsck --full; done`

It now is approximately half as fast, but it does output it's "state".

After this I played around some with the suggestions in this thread and finally got to a point where I could `git stash` and `git stash drop` a lot of the broken stuff.

first problem solved

Afterwards I still had the following problem: `unable to resolve reference 'refs/remotes/origin/$branch': reference broken` which could be solved by `$ rm \repo.git\refs\remotes\origin\${branch}`

```
$ git fetch
```

I then did a `$ git gc --prune=now`

```
$ git remote prune origin
```

for good measure and

```
git reflog expire --stale-fix --all
```

to get rid of `error: HEAD: invalid reflog entry $blubb` when running `git fsck --full`.

edited Sep 10 '17 at 17:38

answered Sep 8 '17 at 13:45



memo42

31 ● 2

I encounter the same problem, and I use a very simple way to fix it. I found that those missing files existed on my teammate's computer.

1

I **copied those files one by one** to a git server (9 files total), and that fixed the problem.



edited Mar 14 '18 at 2:45

answered Mar 14 '18 at 2:24



ndmeiri

4,648 ● 12 ● 31 ● 40



qiucw

41 ● 8

Here is a way to solve the problem if your public repo on github.com is working, but your local repo is corrupt. Be aware that you will loose all the commits you've done in the local repo.





everything from the corrupt repo (except the `.git` folder), and paste it the cloned repo that is working.

This may not be a practical solution (since you delete the local commits), however, you maintain the code and a repaired version control.

Remember to back-up before applying this approach.

answered Jun 12 '18 at 7:22



Code Realistic

53 ● 3



1



My colleagues and I have crossed several times with this same problem and to solve it we simply do the steps that I describe below. It is not the most elegant solution that can be found but it works without loss of data.

1. Rename the current working directory. (`old_project` for this example).
2. Clone the repository within a new directory using `git clone`.
3. On the command line, change the working directory to the newly created project and switch to the branch you have been working on.
4. Copy all files and directories within `old_project` (except the `.git` directory) to the newly created project directory.
5. Check your working tree status (note that there are many more changes than you expect) and then commit the changes.

I hope it helps...

answered Jun 11 '18 at 13:58



ymiraola

100 ● 7



1



In my case, it wasn't important for me to keep the local commit history. So if that applies to you too, you can do this as a quick alternative to the solutions above:

You basically just replace the corrupted `.git/` directory with a clean one.

Lets presume the following directory for your project with the corrupted git: `projects/corrupt_git/`

1. `cp projects/corrupt_git projects/backup` - (optional) make a backup
2. `git clone [repo URL] projects/clean_git` - so that you get `projects/clean_git`
3. `rm -rf corrupt_git/.git/` - remove the corrupted `.git` folder
4. `mv clean_git/.git/ corrupt_git/` - move clean git to `corrupt_git/.git`
5. `git status` in `projects/corrupt_git` - to make sure it worked

answered Nov 1 '19 at 13:17



Benjamin Basmaci



if you have an OLD backup and is in a hurry:

0 make a NEW BACKUP of your current, git-broken, project path.

1. move your `.git` to trash (never delete)
2. copy `.git` from the OLD backup
3. `git pull` (will create merge conflicts)
4. move all your sources (everything you put in git) to trash: `./src` (never delete)
5. .copy all your sources (everything you put in git) from the NEW BACKUP
6. accept all "merges" at `git gui`, push and... clap your hands!

edited Apr 2 '18 at 4:53



phucv

23.8k ● 11 ● 87 ● 325

answered Sep 23 '16 at 4:15



Aquarius Power

2,962 ● 4 ● 25 ● 54

Copy everything (in the folder containing the `.git`) to a backup, then delete everything and restart. Make sure you have the git remote handy:

```
git remote -v
origin git@github.com:rwldrn/idiomatic.js.git (fetch)
origin git@github.com:rwldrn/idiomatic.js.git (push)
```

Then

```
mkdir mygitfolder.backup
cp mygitfolder/* mygitfolder.backup/
cd mygitfolder
rm -r * .git*
git init
git remote add origin git@github.com:rwldrn/idiomatic.js.git
```

Then merge any new files manually, and try to keep your computer turned on.

edited Feb 6 '14 at 17:04

answered May 1 '13 at 4:41



Shelvacu

3,506 ● 18 ● 40

▲ `rm -rf *` might have very bad consequences. Did you really mean that ? — [tommyk](#) Feb 6 '14 at 12:24

▲ @tommyk hence the copy to the backup first. I guess the force isn't necessary. — [Shelvacu](#) Feb 6 '14 at 17:04

I fixed my git error: object file is empty by:

0 1. Saving a copy of all files that I edited since my last successful commit/push,





Hopes this helps.

answered Feb 25 '19 at 21:39



JattCode113

62 ● 7



Actually I had the same problem. Have a copy of your code before trying this.

0

I just did `git reset HEAD~`



my last commit was undone then I committed it again, problem solved!



answered May 1 at 11:40



k441i

21 ● 4



The twelve step solution covered above helped get me out of a jam as well. Thanks. The key steps were to enter:

0



```
git fsck --full
```



and remove all empty objects

```
rm .git/objects/...
```

Then getting the two lines of the flog:

```
tail -n 2 .git/logs/refs/heads/master
```

With the returned values

```
git update-ref HEAD ...
```

At this point I had no more errors, so I made a backup of my most recent files. Then do a git pull followed by a git push. Copied my backups to my git repository file and did another git push. That got me current.

edited Apr 2 '18 at 7:00



rollstuhlfahrer

3,674 ● 8 ● 20 ● 34

answered Jul 10 '16 at 5:43



Jacob

39 ● 6



This also happens to me almost regularly. Haven't made a protocol when this happens exactly, but I have a suspicion that it occurs whenever my virtual machine exists "unexpectedly". If I close the VM window (I am using Ubuntu 18.04) and start again, things always(?) work. But if the VM window is still open when my laptop is shut down (Windows host system), then I encounter this problem rather frequently.

0



By using our site, you acknowledge that you have read and understand our [Cookie Policy](#), [Privacy Policy](#), and our [Terms of Service](#).



1. thank you - they are very useful; I usually save a local copy of my code, restore the repo from remote, and move the backup copy back into the local folder.
2. as the underlying problem is not really a git issue, but rather a VM and/or Linux issue, I wonder if there shouldn't be a way to cure the reason rather the symptoms? Doesn't this kind of error indicate that some file system changes are not "applied" in any reasonable time, but only cached? (see for example <https://unix.stackexchange.com/questions/464184/are-file-edits-in-linux-directly-saved-into-disk>) -- to me it appears as if virtual Linux machines don't fsynch their stuff frequently enough. Whether this is an issue of Oracle's VirtualBox (which otherwise works very nicely) or of the guest file system, or of some settings, which we all overlook, is beyond my expertise. But I would be happy if someone could shed light on this.

answered Mar 7 at 16:17



maschu

1,087 ● 1 ● 12 ● 29

Had the same problem after checking out master from a clean branch. After a while I recognized a lot of modified files in master. I don't know why they have been there, after switching from a clean branch. Anyways, because the modified files made no sense to me, I just stashed them and the error was gone.

git:(master) git stash

answered Jul 1 '14 at 14:40



ownking

1,768 ● 1 ● 19 ● 33

